جامعة الفيصل
Alfaisal University

كلية الهندسة والحوسبة المتقدمة
College of Engineering and Advanced Computing

Introduction to Artificial Intelligence (SE 444)

Lecture 7.3: Propositional Logic
Part 2: Logical Inference and Reasoning

Prof. Anis Koubaa

akoubaa@alfaisal.edu



This lecture provides a comprehensive overview of logical reasoning in AI, focusing on Propositional Logic. In previous lectures, we explored the role of knowledge-based agents and the essentials of propositional logic, including its syntax, semantics, models, truth tables, and the construction of knowledge bases. In this lecture, we introduce key concepts such as entailment vs. implication and differentiate between them, alongside the three fundamental logical relationships: entailment, contradiction, and contingency. We will examine the practical applications of Tell/Ask operations, discuss the crucial concept of satisfiability, and delve into methods for model checking. Finally, the lecture will cover various inference rules, highlighting the properties of soundness and completeness to equip you with a robust understanding of intelligent reasoning systems.

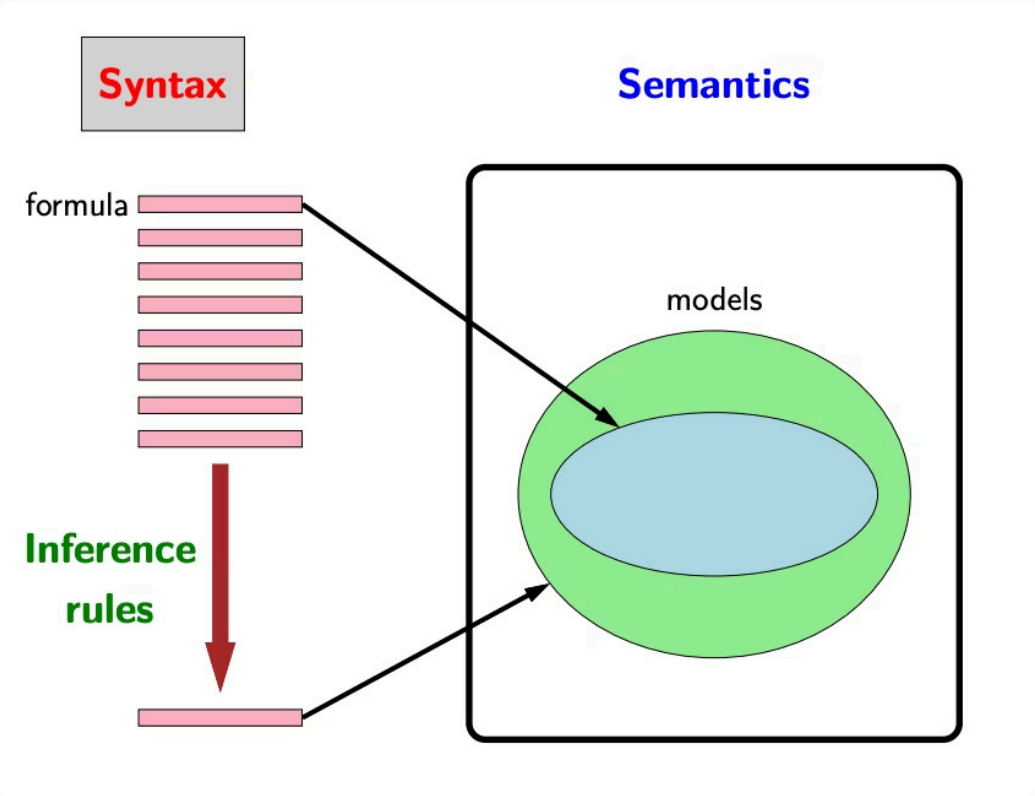
Recap: Knowledge-Based Agents

🧠 Knowledge-Based Approach

- 🧩 **Core Idea:** Agents use a **Knowledge Base (KB)** to **represent** and **reason** about the world.
- 🧱 Maintain **internal models** of their environment for informed decision-making.
- ⚡ **Unlike reactive agents**, they don't just react – they **reason** before acting.
- 🎯 Accept **explicitly defined goals** and plan actions to achieve them.
- 🧠 Gain competence by:
  - 📘 **Being told** new knowledge (declarative input).
  - 🔍 **Learning** from experience and **inference**.
- 🚀 Result: **Smarter, adaptable agents** that can handle new tasks efficiently.



Recap: Propositional Logic Essentials



Reference:
https://web.stanford.edu/class/archive/cs/cs221/cs221.1186/lectures/logic1.pdf

- **Logic as Search for Truth**
  We start from *known facts (KB)* and apply *inference rules* to discover new truths.
- **Three Ingredients of Logic**
  - **Syntax:** defines valid formulas (e.g., $(P \wedge Q) \rightarrow R$
  - **Semantics:** assigns truth via **models** (possible worlds)
  - **Inference Rules:** derive new formulas true in all models
- **Models as Possible Worlds**
  Each model = a complete truth assignment (e.g., `Rain=1, Wet=0`).
  Formulas represent sets of models where they hold true.
- **Interpretation Function** $I(f, w)$
  Evaluates whether formula f is **true** in model w.
  Recursive definition mirrors syntax.
- **Knowledge Base (KB)**
  A set of formulas → their **intersection of models**:
  $M(KB) = \bigcap_{f \in KB} M(f)$
  More formulas ⇒ fewer possible worlds ⇒ more specific knowledge.

✨ Entailment: Already Known Information

Definition: KB **entails** f
(written KB f) iff M(f) ⊇ M(KB).

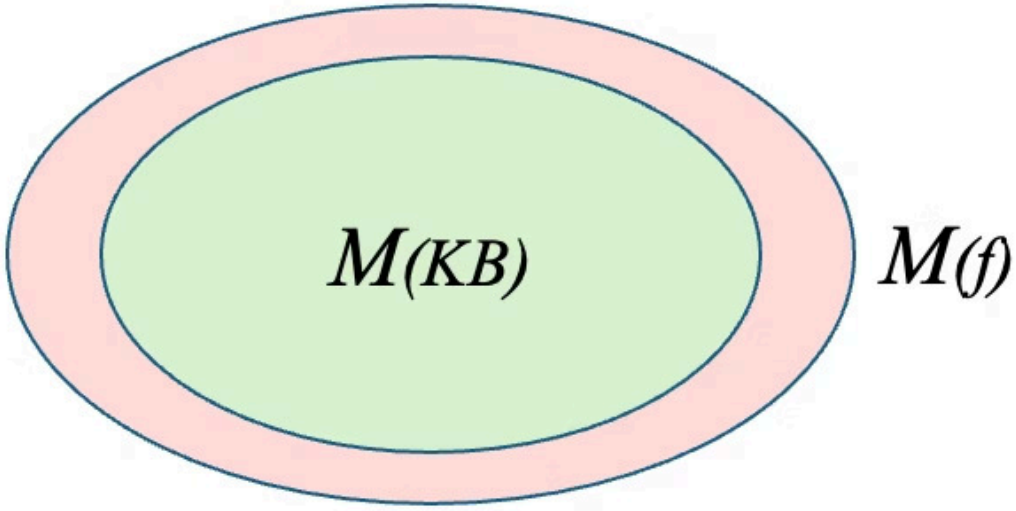
🎯 Intuition

Formula f adds **no new information/constraint**
→ it was already known from KB

📝 Example

Rain ∧ Snow Snow

If it's raining AND snowing, then obviously it's snowing!



✨ Entailment vs. Implication

Implication (→)

- *Part of the language* ***(syntax)*** – a logical **connective** within formulas.
- Example: $(Rain \land Snow) \rightarrow Snow$ is itself a **formula**.
- It expresses a **relationship between two propositions** inside logic.
- Its truth depends on the **values** of its components in a **model**.
- Evaluated by the **truth table** of implication.

Entailment (□)

- *Meta-level* ***(semantics)*** – a **relationship between formulas or KBs**.
- $KB \models f$ means: *every model that satisfies KB also satisfies f.*
- Expresses **logical consequence** – f is true in all worlds where KB is true.
- Not written *inside* logic; it's about what follows logically *from* the logic.
- Tested via **model checking** or **proof derivation**.

Modus Ponens: Linking Implication and Entailment

Given formulas (Implication inside logic)

$(Rain \rightarrow Wet), \quad Rain$

- “If it rains, then it’s wet.”
- “It is raining.”

Inference step (Entailment between formulas):

From these, we can ***logically*** *conclude*:

$\{(Rain \rightarrow Wet),\ Rain\} \models Wet$

⇒ Every model that satisfies both premises also satisfies “Wet.”

Interpretation in words:

- The **implication (→)** connects two propositions ***within*** the logic.
- The **entailment (⊨)** connects *sets of formulas* – it states that “Wet” *must* follow logically from the knowledge base.

💡 Key Idea:

> **Implication** expresses the rule ***inside*** logic.
> **Entailment** expresses the ***result*** of reasoning across all models.

🧩 Entailment vs. Implication

Aspect	Implication (→)	Entailment (⊨)
Level	Inside logic (syntactic)	Outside logic (semantic relation)
Meaning	“If P then Q”	“Q follows logically from P”
Evaluated by	Truth table in a single model	All models satisfying KB
Type	Formula	Meta-relation between formulas
Example	$(Rain \land Snow) \rightarrow Snow$	$Rain \land Snow \models Snow$

🧩 Mathematical Proof by Entailment

Theorem: If n is an odd number, then n^2 is odd.

This example elegantly demonstrates how a mathematical proof aligns with the principles of logical inference and entailment.

Step 1: Define Propositions

We translate the statements into logical propositions:

$P : n \text{ is odd}$

$Q : n^2 \text{ is odd}$

Our goal is to show that P entails Q, written as $P \models Q$.

Step 2: Establish the Arithmetic Fact (Axiom)

From basic number theory, an odd number n can be expressed as $n = 2k + 1$ for some integer k. Let's square this expression:

$$n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$$

Since $2k^2 + 2k$ is an integer, n^2 fits the form $2(\text{integer}) + 1$, which proves that if n is odd, then n^2 is also odd.

This mathematical derivation establishes the implication: $P \to Q$.

Step 3: Apply Logical Reasoning

We now have a Knowledge Base (KB) containing our initial premise and the derived implication:

$$KB = \{P, (P \to Q)\}$$

By the rule of **Modus Ponens**, which states that if we have A and $A \to B$, we can infer B. In our case, this means:

$P, (P \to Q) \vdash Q$

Step 4: Express as Entailment

The logical inference from Step 3 can be formally stated as an entailment:

$$\{P, (P \to Q)\} \models Q$$

✅ This means that in **every model** (or possible world) where both "n is odd" and "if n is odd then n^2 is odd" are true, it necessarily follows that "n^2 is odd" must also be true.

Entailment Examples Across Domains



Healthcare Expert System

$\{\text{Fever}, \text{Cough}, (\text{Fever} \land \text{Cough} \rightarrow \text{Flu})\} \models \text{Flu}$

If the patient has fever and cough, the system infers “Flu.”

$\{\text{HighBP}, (\text{HighBP} \rightarrow \text{Risk_of_Stroke})\} \models \text{Risk_of_Stroke}$



Legal Expert System

$\{\text{Contract}, \text{Signed}, (\text{Contract} \land \text{Signed} \rightarrow \text{LegallyBinding})\} \models \text{LegallyBinding}$

A signed contract entails a legally binding agreement.

$\{\text{Theft}, (\text{Theft} \rightarrow \text{Crime})\} \models \text{Crime}$



Education Tutoring System

$\{\text{CompletedLesson1}, (\text{CompletedLesson1} \rightarrow \text{UnlockLesson2})\} \models \text{UnlockLesson2}$

Finishing one lesson entails unlocking the next.



Financial / Business Rule System

$\{\text{Income} > 100\text{K}, (\text{Income} > 100\text{K} \rightarrow \text{HighTaxBracket})\} \models \text{HighTaxBracket}$

Simple Medical Expert System (Python)

This Python example demonstrates logical entailment in a rule-based expert system. It uses a **Knowledge Base** (implication rules) and patient **Facts** (symptoms) to infer a diagnosis.

```
# 1. Knowledge Base (KB) - Rules as Implications
KNOWLEDGE_BASE = {
    "Flu": {"fever", "cough", "body_aches"},
    "Common Cold": {"cough", "runny_nose"},
    "Allergies": {"sneezing", "itchy_eyes"},
    "Strep Throat": {"fever", "sore_throat", "swollen_tonsils"},
}

def medical_expert_system(symptoms, rules):
    """Infers diagnosis based on symptoms and KB."""
    patient_symptoms = set(symptoms)
    for diagnosis, required_symptoms in rules.items():
        # Entailment check: If all required symptoms are present in patient_symptoms
        if required_symptoms.issubset(patient_symptoms):
            return diagnosis
    return "Unknown condition"

# --- Example Usage ---
# 2. Facts - Patient's known symptoms
patient_1_symptoms = ["fever", "sore_throat", "runny_nose", "swollen_tonsils"]

# 3. Inference (Entailment) - Run the system
diagnosis_1 = medical_expert_system(patient_1_symptoms, KNOWLEDGE_BASE)

print(f"Patient Symptoms: {patient_1_symptoms}")
print(f"Diagnosis: {diagnosis_1}") # Expected: Strep Throat
```

How it relates to Entailment (KB □ f):

- **Knowledge Base (KB):** The `KNOWLEDGE_BASE` dictionary represents our logical implication rules (e.g., `(Fever ∧ SoreThroat ∧ SwollenTonsils) → StrepThroat`).
- **Facts:** `patient_symptoms` are the known premises.
- **Inference:** The `issubset()` check determines if the premises for a diagnosis are met. If so, the diagnosis is **entailed**.

Formally: $\{ \text{KB_rules, Patient_Facts} \} \models \text{StrepThroat}$

Simple Medical Expert System (Python)

Here's a **simple Python example** of a *rule-based medical expert system* that mimics the logic shown in your image – e.g., reasoning like "If the patient has fever and cough, then flu is inferred."

🩺 Python Code

```
# Simple Rule-Based Medical Expert System
# Author: Example for Educational Use

def diagnose(symptoms):
    """Diagnose based on simple logical entailment rules."""

    # Define rules (as entailments)
    rules = [
        ({"fever", "cough"}, "flu"),
        ({"fever", "rash"}, "measles"),
        ({"headache", "nausea"}, "migraine"),
        ({"high_bp"}, "risk_of_stroke"),
    ]

    inferred_diseases = set()

    for condition, disease in rules:
        if condition.issubset(symptoms):
            inferred_diseases.add(disease)

    return inferred_diseases


# Example usage:
if __name__ == "__main__":
    # Example 1: Patient with fever and cough
    patient_symptoms = {"fever", "cough"}
    result = diagnose(patient_symptoms)
    print("Patient symptoms:", patient_symptoms)
    print("Inferred diagnosis:", result)

    # Example 2: Patient with headache and nausea
    patient_symptoms = {"headache", "nausea"}
    result = diagnose(patient_symptoms)
    print("\nPatient symptoms:", patient_symptoms)
    print("Inferred diagnosis:", result)
```

🧠 Explanation

- The system defines **rules** of the form `{conditions} → conclusion` (like logical entailment).
- The function checks whether the patient's symptoms satisfy any rule's condition.
- If so, it **infers** the corresponding disease or risk.

💡 Example Output

```
Patient symptoms: {'fever', 'cough'}
Inferred diagnosis: {'flu'}

Patient symptoms: {'headache', 'nausea'}
Inferred diagnosis: {'migraine'}
```

💥 Contradiction: Conflicting Information

Definition: A Knowledge Base (KB) **contradicts** a formula f if, in every model where KB is true, f is false.

This can be written as $KB \models \neg f$.

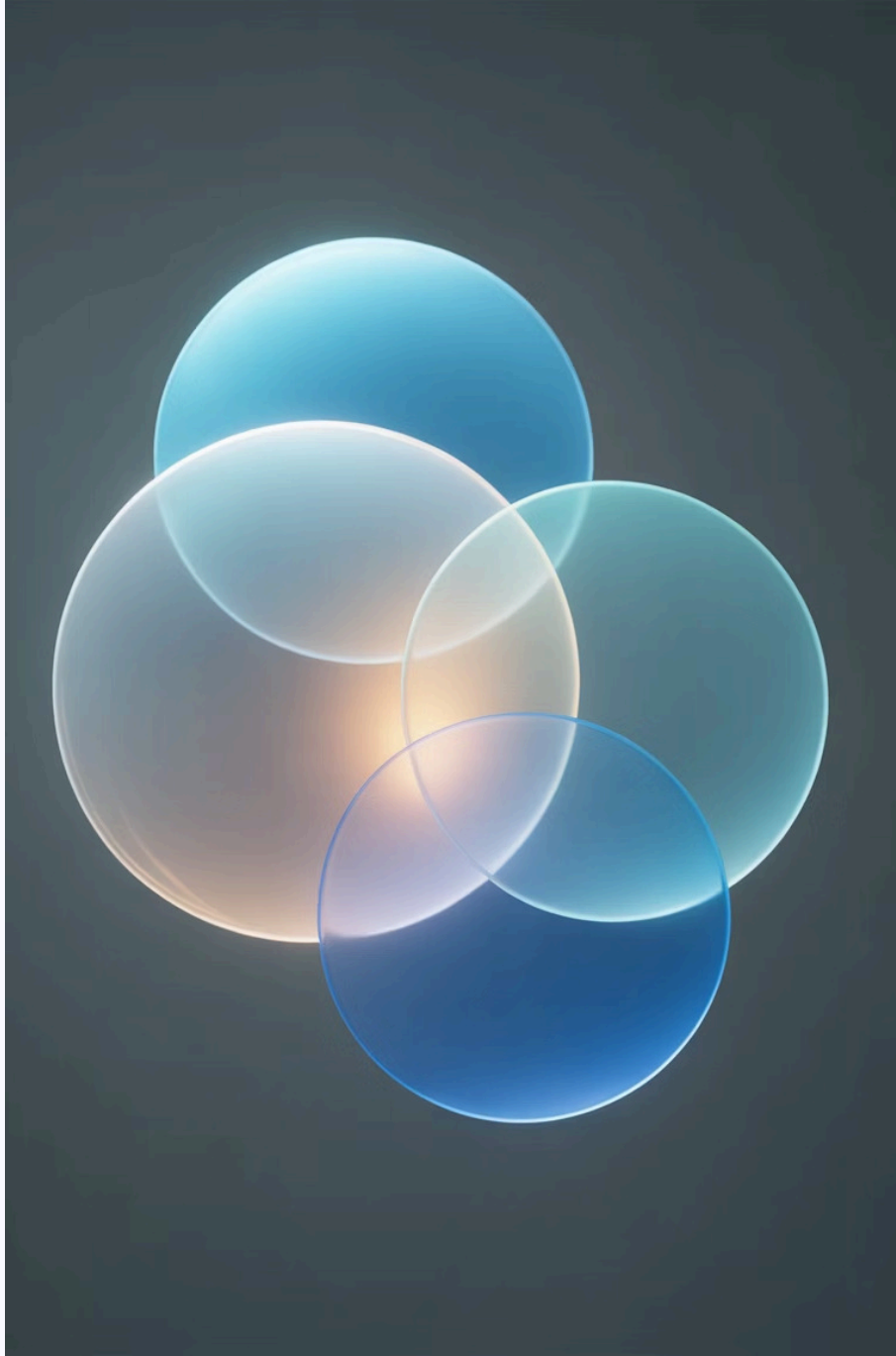
🎯 Intuition

The formula f presents information that is **inconsistent** with what the Knowledge Base already asserts. If the KB is true, f simply cannot be true.

📝 Example

If $KB = \{P, (P \rightarrow Q)\}$, we know $KB \models Q$. Therefore, if we consider the formula $\neg Q$, then KB contradicts $\neg Q$.

A contradictory KB (e.g., $\{P, \neg P\}$) has no models, meaning it is inherently false and entails everything.



✨ Contradiction: Conflicting Information

Definition: KB **contradicts** f

$$\mathcal{M}(KB) \cap \mathcal{M}(f) = \varnothing$$

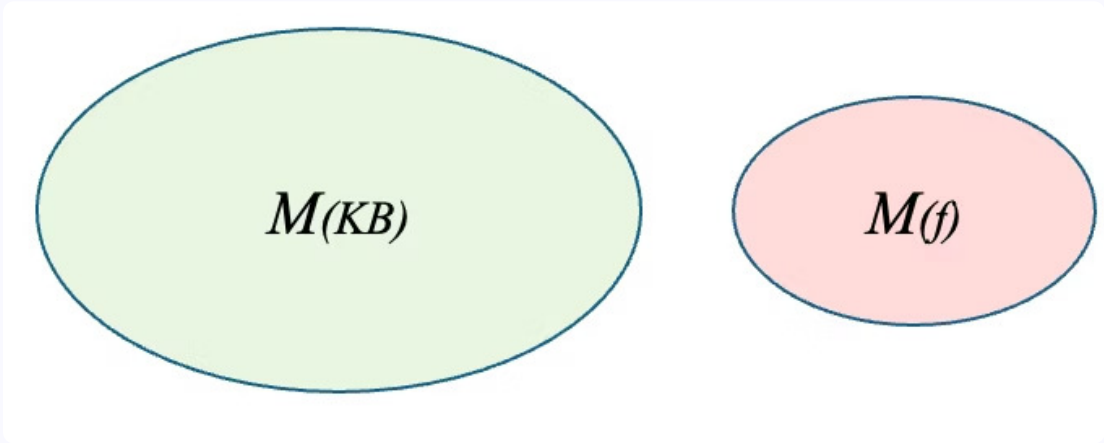
📝 Intuition

There is **no possible model** (no world) in which both KB and f are true together.
→ They cannot be satisfied simultaneously.

✅ Example:
If $KB = \{Rain\}, \quad f = \neg Rain$

then $\mathcal{M}(KB) \cap \mathcal{M}(f) = \varnothing$

so KB and f **contradict** each other.



💥 Contradiction: Conflicting Information

Healthcare Expert System

$\{\text{Fever}, \neg\text{Fever}\}$ A patient cannot both have and not have a fever.

Legal Expert System

$\{\text{Minor}, (\text{Minor} \rightarrow \neg\text{EligibleForContract}), \text{EligibleForContract}\}$
A minor cannot be both ineligible and eligible for a contract simultaneously.

$\{\text{Innocent}, \text{Guilty}\}$ A person cannot be both innocent and guilty of the same charge.

Education Tutoring System

$\{\text{PassedExam}, \neg\text{PassedExam}\}$ A student cannot have both passed and not passed the same exam.

$\{\text{CompletedCourse}, \neg\text{EligibleForCertificate}\}$ If completion implies eligibility, these facts contradict each other.

Financial / Business Rule System

$\{\text{Balance} < 0, \text{Balance} \geq 0\}$ An account balance cannot be simultaneously negative and non-negative.

$\{\text{TaxExempt}, \text{TaxApplied}\}$ An entity cannot be both tax-exempt and have tax applied under the same conditions.

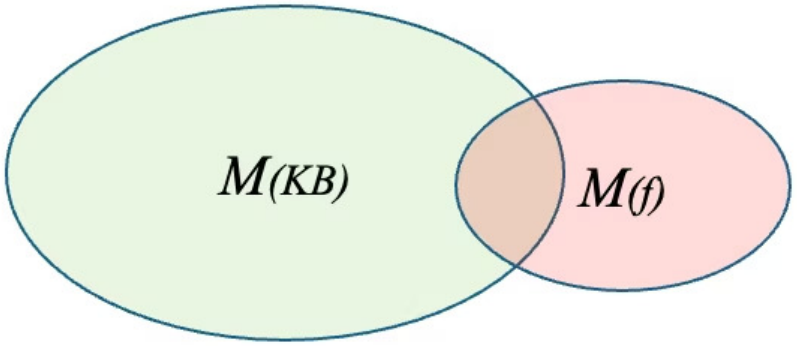
🤔 Contingency (Non-Trivial Information)

Definition: A formula f is **contingent** with respect to a Knowledge Base (KB) if it neither contradicts nor is fully entailed by KB.

$$\emptyset \subsetneq \mathcal{M}(KB) \cap \mathcal{M}(f) \subsetneq \mathcal{M}(KB)$$

- f adds **non-trivial information** to the knowledge base (KB).
- There is **some overlap**, but **not total containment** between models of KB and models of f.

$$\emptyset \subsetneq \mathcal{M}(KB) \cap \mathcal{M}(f) \subsetneq \mathcal{M}(KB)$$



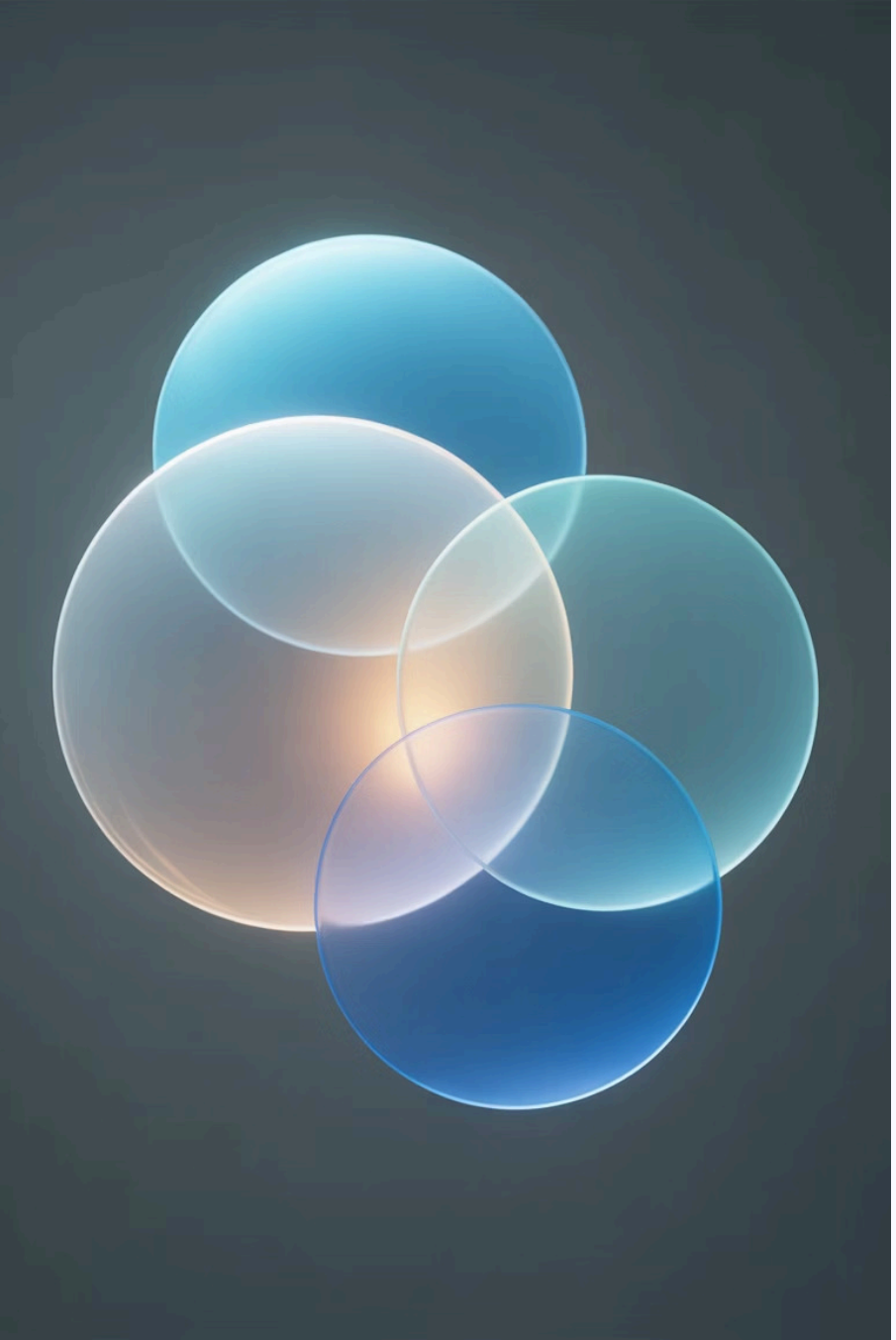
📝 Example

Given a KB about weather, consider these formulas:

$KB = \{\text{Rain}\}$, $f = \text{Snow}$

This means f is partially consistent with KB – true in some worlds where KB is true, but not all.

> 💡 **Key Idea:** Contingency represents a partial overlap of models, meaning f enriches KB without causing contradiction or introducing redundancy.



🤔 Contingency

New Information

🎯 Condition

$$\emptyset \subsetneq \mathcal{M}(\text{KB}) \cap \mathcal{M}(f) \subsetneq \mathcal{M}(\text{KB})$$

💡 Intuition

Formula f adds **non-trivial information** to KB

📝 Example

Given KB about weather, adding "Traffic" is contingent - it could be true or false

🤔 Contingency Examples Across Domains

Healthcare Expert System

$KB = \{\text{Fever}\}, f = \text{Cough}$

Some patients with fever cough, some don't – showing partial overlap.

$KB = \{\text{Diabetic}\}, f = \text{HighBP}$

Not all diabetics have high blood pressure, but some do.

Legal Expert System

$KB = \{\text{ContractSigned}\}, f = \text{DisputedContract}$

Some signed contracts are disputed, some are not.

$KB = \{\text{Employee}\}, f = \text{UnionMember}$

Some employees are union members; not all.

Education Tutoring System

$KB = \{\text{AttendedClass}\}, f = \text{PassedExam}$

Some students who attend pass; others may not.

$KB = \{\text{SubmittedAssignment}\}, f = \text{HighGrade}$

Submission helps but doesn't guarantee a high grade.

Financial / Business Rule System

$KB = \{\text{HighIncome}\}, f = \text{OwnsHouse}$

Many high-income people own homes, but not all.

$KB = \{\text{StartupFounder}\}, f = \text{ProfitableBusiness}$

Some startups are profitable, others are not.

Contradiction and Entailment

🔁 Entailment–Contradiction Connection

- **Contradiction**

$$KB \textbf{ contradicts } f \textbf{ iff } \mathcal{M}(KB) \cap \mathcal{M}(f) = \varnothing$$

→ No model satisfies both KBKB and f.

- **Entailment**

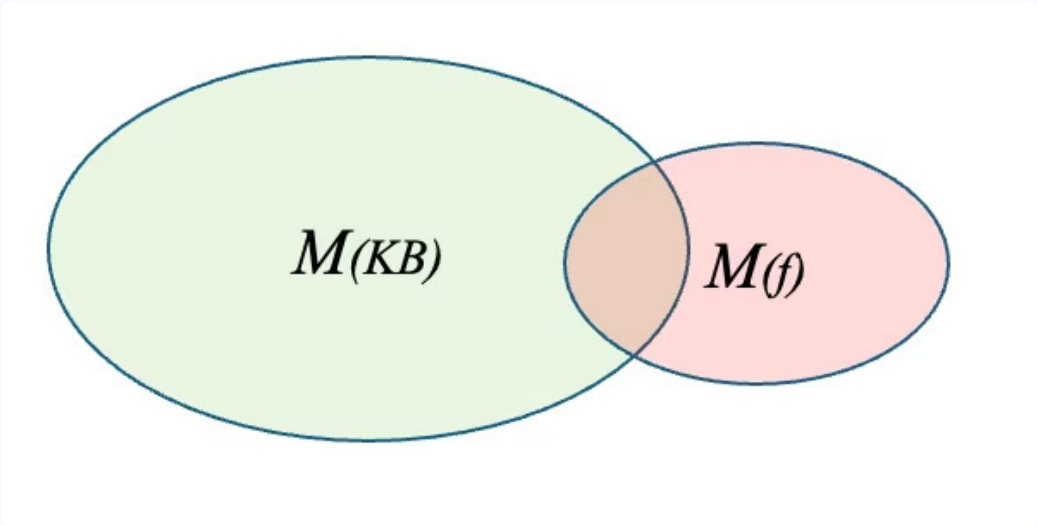
$$KB \text{ entails } \neg f \textbf{ iff } \mathcal{M}(\neg f) \supseteq \mathcal{M}(KB)$$

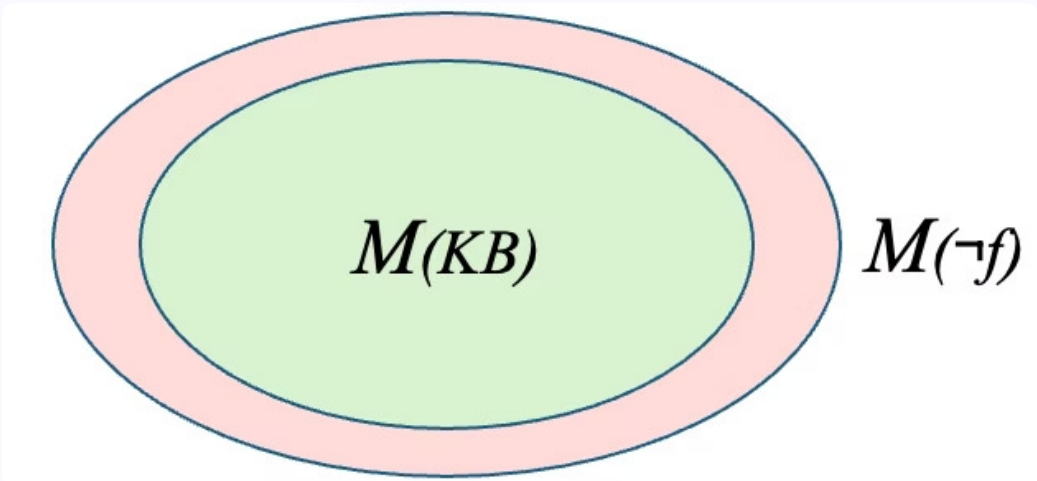
→ Every model of KB also satisfies the negation of f.

- **Proposition:** $KB \text{ contradicts } f \;\Leftrightarrow\; KB \models \neg f$

→ Contradiction and entailment are logically equivalent perspectives.

- **Reason**
  $\mathcal{M}(f)$ and $\mathcal{M}(\neg f)$ **partition the space** of all possible models – every world makes exactly one of them true.





💬 Tell Operation: Adding Facts

$$\text{Tell}[f] \longrightarrow \text{KB} \longrightarrow ?$$

Example: Tell: "It is raining."

01

✅ Already Knew That

Entailment: KB ▯ f

Response: "I already knew it was raining."

02

❌ Don't Believe That

Contradiction: KB ▯ ¬f

Response: "That contradicts what I know."

03

🆕 Learned Something New

Contingent: Update KB

Response: "Thanks, I've learned something new!"

❓ Ask Operation: Querying Knowledge

$$\text{Ask}[f] \longrightarrow \text{KB} \longrightarrow ?$$

Example: Ask: "Is it raining?"

01

✅ Yes

Entailment: KB ▯ f

Response: "Yes, it is definitely raining."

02

❌ No

Contradiction: KB ▯ ¬f

Response: "No, it is definitely not raining."

03

🤷 I Don't Know

Contingent: Insufficient information

Response: "I don't have enough information to determine that."



🔍 Satisfiability: The Universal Tool

> **Definition:** A knowledge base KB is **satisfiable** if M(KB) ≠ ∅

We can reduce **Ask[f]** and **Tell[f]** to satisfiability checking:

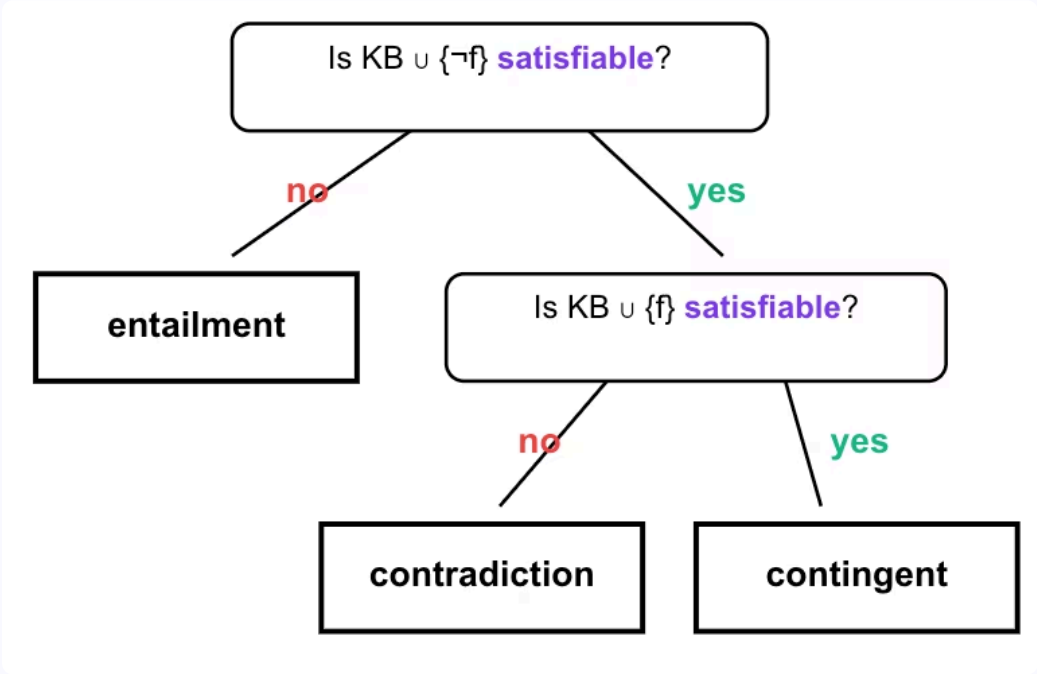
- **Entailment check:** Is KB ∪ {¬f} satisfiable? If no → KB ⊨ f
- **Contradiction check:** Is KB ∪ {f} satisfiable? If no → KB contradicts f

🔍 Satisfiability: The Universal Tool

Definition: A knowledge base KB is **satisfiable** if `M(KB) ≠ ∅`

We can reduce **Ask[f]** and **Tell[f]** to satisfiability checking:

- **Entailment check:** Is `KB ∪ {¬f}` satisfiable? If no → `KB  f`
- **Contradiction check:** Is `KB ∪ {f}` satisfiable?
  If no → KB contradicts f
  if yes → **Contingent** – both $KB \cup \{f\}$ and $KB \cup \{\neg f\}$are consistent.



Both $KB \cup \{f\} KB \cup \{\neg f\}$ can be satisfiable.

Let $KB = \{Rain\}$ and $f = Snow$

Then:

- $KB \cup \{f\} = \{Rain, Snow\}$ → **satisfiable** (possible world where both happen).
- $KB \cup \{\neg f\} = \{Rain, \neg Snow\}$ → also **satisfiable** (possible world where it rains but no snow).

✅ Both consistent → **contingent relationship**.

🚀 Summary

Case	Meaning	Models
Both $KB \cup \{f\}$ and $KB \cup \{\neg f\}$ satisfiable	**Contingent**	f adds partial new info
Only $KB \cup \{\neg f\}$ unsatisfiable	**Entailment**	f must be true
Only $KB \cup \{f\}$ unsatisfiable	**Contradiction**	f cannot be true

🧩 Model Checking = CSP Solving

Definition: Model Checking

Model checking is the process of determining whether a formula f (or a knowledge base KB) is **satisfiable** – i.e., whether there exists a **model** (truth assignment) that makes it **true**.

- **Input:** knowledge base KB
- **Output:** does it exist a satisfying model `M(KB) ≠ 0`?

Key Points

- **Goal:** Find if there exists a model w such that $I(f, w) = \text{True}$
- **If such a model exists:** formula is **satisfiable (SAT)**.
- **If no model exists:** formula is **unsatisfiable (UNSAT)**.

Relation to Constraint Satisfaction Problems (CSPs)

Checking satisfiability (SAT) is a **special case** of solving **CSPs**.

Both involve:

- **Variables** (propositions)
- **Domains** (True/False values)
- **Constraints** (logical formulas)

Example

For $f = (\text{Rain} \vee \text{Wet}) \wedge \neg(\text{Rain} \wedge \text{Wet})$:

Model checking tests **all truth assignments** for Rain, Wet to find if any satisfy f.

✅ **Summary:** Model checking = exploring all possible truth assignments (models) to verify **satisfiability** – the logical analog of solving a CSP.

🧩 Model Checking = CSP Solving

1	2	3
🧱 Logic World	🔄 Mapping	🧩 CSP World
• Propositional symbol	⇒	• Variable
• Formula	⇒	• Constraint
• Model	⇐	• Assignment

Checking satisfiability (SAT) in propositional logic is a **special case of solving CSPs**!

⚙️ Popular SAT Algorithms

- 🧩 **DPLL (Backtracking Search)**
  - Systematic **backtracking search** algorithm: Tries all possible truth assignments.
  - Uses **unit propagation** and **pure literal elimination** to prune the search space: Uses smart pruning to skip impossible branches.
  - Guarantees completeness – finds a solution or proves UNSAT.
    - **Complete** → always finds a solution or proves none exists.

- 🎲 **WalkSAT (Random Search)**
  - **Randomized local search** algorithm. Starts with a random guess for all variables.
  - Starts with a random assignment and iteratively flips variables to reduce conflicts. Flips values to reduce conflicts (makes more formulas true).
  - Fast for large problems but **incomplete** – may not find all solutions: **Fast**, but may miss some solutions (not complete).

💡 **Summary:**

> **DPLL** = logical, exhaustive search (complete)
> **WalkSAT** = probabilistic, heuristic search (fast but incomplete)

🔧 Model Checking Example

$$\text{KB} = \{A \vee B, B \leftrightarrow \neg C\}$$

🔢 CSP Variables

{A, B, C}

Each can be 0 or 1

⚡ CSP Constraints

A ∨ B must be true

B ↔️ ¬C must be true

✅ Solution

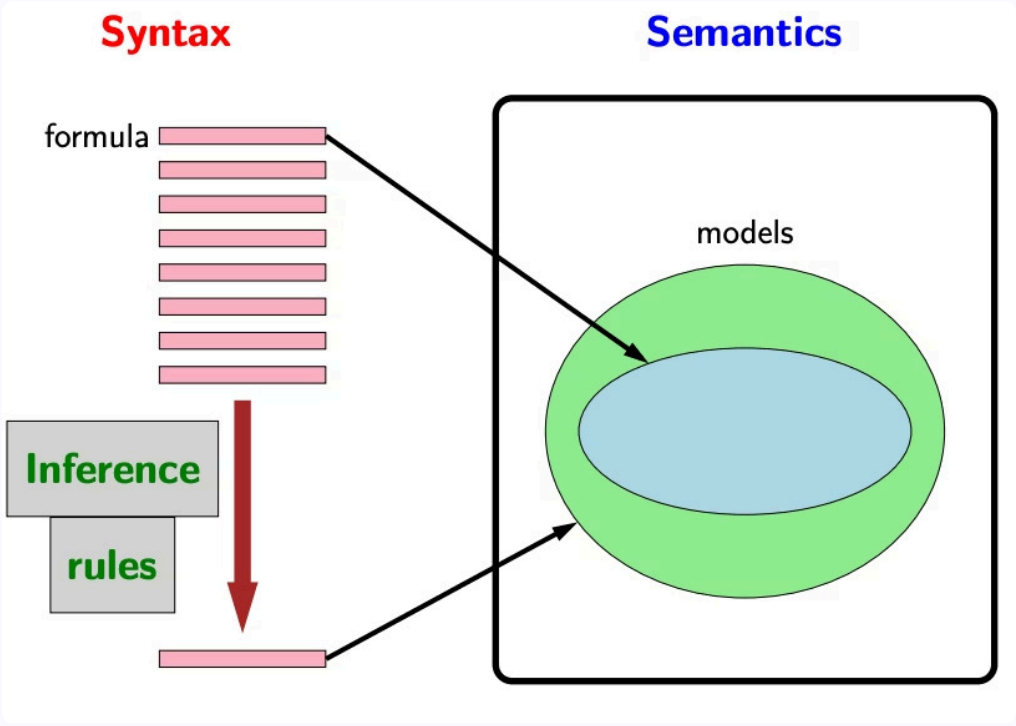
{A:1, B:0, C:1}

This is a satisfying model!

Example of SAT/Model Checking Problem

- **Knowledge Base (KB):** $KB = \{A \vee B,\ B <=> \neg C\}$ → These are the logical constraints (rules).
- **Propositional Symbols (CSP Variables):** $\{A, B, C\}$
- **CSP Representation:**
  - Each formula acts like a **constraint** between variables.
  - $A \vee B$: at least one of A or B must be True.
  - $B <=> \neg C$: B and C must have **opposite** truth values.
- **Goal (Model Checking):** Find a **truth assignment** that satisfies **all constraints**.
- **Satisfying Assignment (Model):** $\{A : 1,\ B : 0,\ C : 1\}$ ✅ This assignment makes both formulas in KB true.

Inference Rules: The Engine of Logical Reasoning



Reference:
https://web.stanford.edu/class/archive/cs/cs221/cs221.1186/lectures/logic1.pdf

- **Purpose:**
  Apply systematic rules to **derive new truths** from known formulas in the **Knowledge Base (KB)**.
- **Definition:**
  An **inference rule** specifies how to derive a new formula g that **logically follows** from existing formulas $f_1, f_2, \ldots, f_n$.
  $f_1, f_2, \ldots, f_n \vdash g$
- **Goal:**
  Ensure that all derived conclusions are **true in all models** – this property is called **soundness**.
- **Examples:**
  - **Modus Ponens:** From P and $P \to Q$, infer Q.
  - **Conjunction:** From P and Q, infer $P \wedge Q$.
  - **Simplification:** From $P \wedge Q$, infer P.
- **Intuition:**
  Inference rules connect **syntax** (formulas) with **semantics** (truth in models) – they guide the search for truth by expanding what is known.

The Turnstile Symbol (⊢): Syntactic Derivation

Meaning of the Turnstile Symbol (⊢)

$$f_1, f_2, \ldots, f_n \vdash g$$

This is read as **“(g) is derivable from (f_1, f_2, ..., f_n)”** or **“(g) can be proven syntactically from these formulas using inference rules.”**

Example:
From $P \to Q$ and P,
we can **derive** Q:

$$P \to Q,\ P \vdash Q$$

(by *Modus Ponens* inference rule).

Differentiating Logical Symbols

⚖️ Key Differences from Related Symbols

$\rightarrow$	**Implication**	*Inside logic (syntax)*	Connects two formulas (e.g., $P \rightarrow Q$)
$\models$	**Entailment**	*Semantic*	$KB \models f$: Every model that satisfies KB also satisfies f.
$\vdash$	**Derivation / Provability**	*Syntactic*	$KB \vdash f$: f can be **derived** using inference rules.

💡 In Short:

Provable (□)

A formula is **provable** from a Knowledge Base using inference rules.

True in all Models (□)

A formula is **true in all models** that satisfy the Knowledge Base (semantic concept).

In a **sound and complete system**, these two concepts align:

$$KB \vdash f \quad \text{if and only if} \quad KB \models f$$

✅ This means everything that can be proven is true, and everything that is true can be proven.

⚡ Inference Rules

Classic Example: Making an inference

> **Fact 1.** It is raining. (Rain)
>
> **Rule 1.** If it is raining, then it is wet. (Rain → Wet)
>
> **Therefore,** it is wet. (Wet)

$$\frac{\text{Rain}, \quad \text{Rain} \rightarrow \text{Wet}}{\text{Wet}}$$

$\frac{\text{Rain}, \quad \text{Rain} \rightarrow \text{Wet}}{\text{Wet}}$	$\frac{Premises}{Conclusion}$

This is **Modus Ponens** - the classic inference rule!



🎯 Modus Ponens: The Foundation

Modus Ponens Inference Rule: For any propositional symbols p and q:

$$\frac{p, \quad p \to q}{q}$$

🌧️ Premise 1

"It is raining"

➡️ Premise 2

"If it rains, then it's wet"

💧 Conclusion

"Therefore, it is wet"

This captures the fundamental **if-then reasoning pattern**!

🏗️ General Inference Framework

Definition: If $f_1, \ldots, f_k, g$ are formulas, then this is an **inference rule**:

$$\frac{f_1, \ldots, f_k}{g}$$

Key Idea: Rules operate directly on **syntax**, not **semantics**!

Inference rules work like grammar rules in logic.
They don't understand what the symbols *mean* –
they just show how to **combine formulas correctly**
so that the result is always **logically valid** (true in every case).
(soundness).

01

📝 Premises

$f_1, f_2, \ldots, f_k$ (what we know)

02

➡️ Rule Application

If premises are in KB, apply rule

03

🎯 Conclusion

Add g to KB (what we derive)



🔄 Forward Inference Algorithm

Algorithm: Forward Inference
Input: set of inference rules Rules

Repeat until no changes to KB:
  Choose formulas $f_1, \ldots, f_k \in KB$
  If matching rule exists: $f_1,\ldots,f_k/g$
    Add g to KB

Definition: KB **derives/proves** f (KB ⊢ f) iff f eventually gets added to KB

🌧️ Inference Example: Chain of Reasoning

Starting KB: {Rain, Rain → Wet, Wet → Slippery}

1

Step 1: Apply Modus Ponens

Rain + (Rain → Wet) ⊢ **Wet**

KB = {Rain, Rain → Wet, Wet → Slippery, Wet}

2

Step 2: Apply Modus Ponens Again

Wet + (Wet → Slippery) ⊢ **Slippery**

KB = {Rain, Rain → Wet, Wet → Slippery, Wet, Slippery}

3

Step 3: Converged

No more rules can be applied

Cannot derive: ¬Wet, Rain → Slippery

*Because **Modus Ponens** only lets you derive facts that are **currently true**, not new conditional statements (implications).*

What Makes Good Inference Rules?

🎭 Semantics

Defines what's **true** in the world (objective truth).

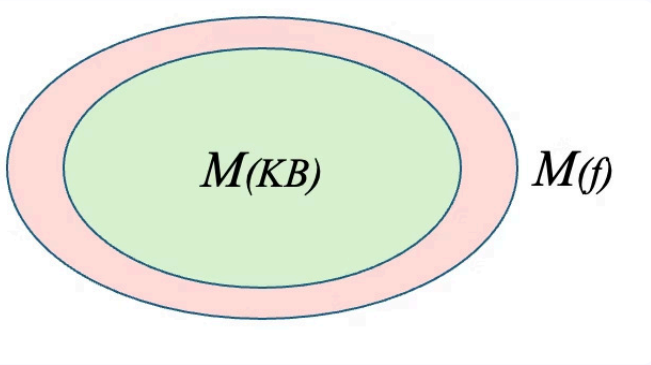
Notation: : $KB \models f$

means **formula f is true in all models** that satisfy KB.

The set of all such truths is: $\{f : KB \models f\}$

📝 Syntax

- Defines what we can **derive** using **inference rules** (like Modus Ponens).
- Notation: $KB \vdash f$ means **we can prove f** from KB using the rules.
- The set of all **derivable** formulas is: $\{f : KB \vdash f\}$



The Big Question

> Do our **inference rules** actually capture **all the true things** – and **only the true things**?

So we ask: $\{f : KB \models f\} \stackrel{?}{=} \{f : KB \vdash f\}$

🧠 In Other Words

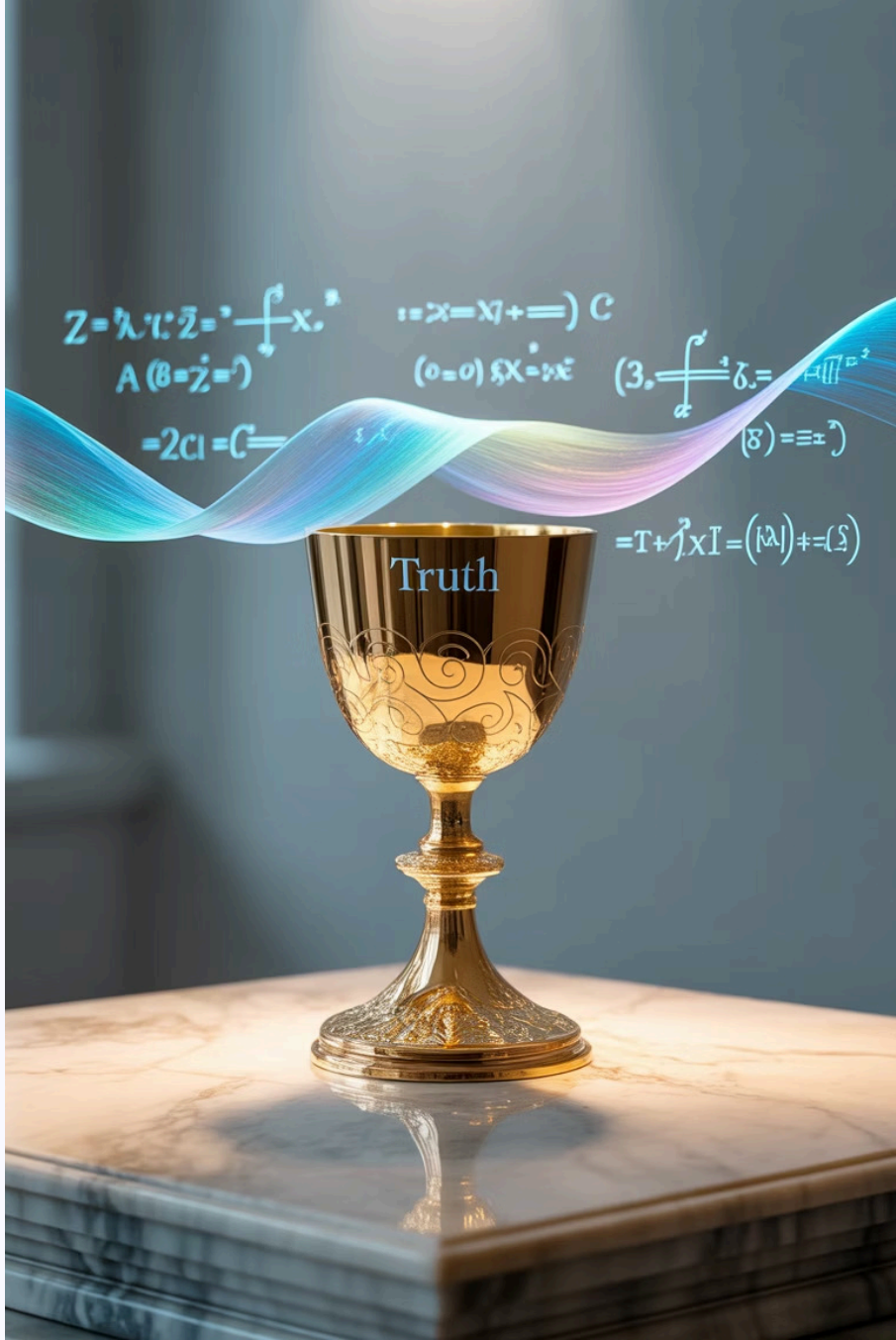
- **Soundness:** Every derivable formula is true
  → If $KB \vdash f$, then $KB \models f$.
- **Completeness:** Every true formula is derivable
  → If $KB \models f$, then $KB \vdash f$.

🏆 Truth: The Gold Standard

Imagine a glass representing the set of formulas entailed by KB:

$$\{f : \mathrm{KB} \models f\}$$

By applying inference rules, we are **filling up the glass** with derived formulas. But how well do we fill it?



🔒 Soundness

Nothing But the Truth

Soundness = No Lies

Definition: A set of inference rules is **sound** if:

$$\{f : \mathrm{KB} \vdash f\} \subseteq \{f : \mathrm{KB} \models f\}$$

Meaning of Soudness

Soundness means:

> **Whatever we can prove with our rules is actually true in the real world.**

In other words: ✅ **No false conclusions are ever derived**.

Intuition: We never overflow the glass - every derived formula is actually true. No false positives!





📈 Completeness

The Whole Truth

Definition: A set of inference rules is **complete** if:

$$\{f : \text{KB} \vdash f\} \supseteq \{f : \text{KB} \models f\}$$

Meaning of Completeness

Completeness means:

> **If something is true in the world, our logic rules can prove it.**

So we don't *miss* any truths.

Intuition: We fill the glass to the brim - every true formula can be derived. No false negatives!

🧩 Four Combinations of Soundness & Completeness

Exploring the various scenarios that arise when considering the soundness and completeness of an inference system.

Type	Description	Student Analogy 💬	Behavior
✅ Sound & Complete	All provable statements are true, and all true statements can be proved.	🧠 The student answers only correct questions and never misses any.	**Ideal system** (e.g., Propositional Logic).
❌ Not Sound but Complete	Proves everything, including false things.	😅 The student answers “yes” to every question, getting all truths but also falsehoods.	**Useless** – can’t trust results.
✅ Sound but Not Complete	Only proves true statements, but misses some truths.	🤔 The student is always correct, but sometimes stays silent (doesn’t know some true answers).	**Safe but limited** – e.g., First-Order Logic.
❌ Not Sound, Not Complete	Proves false things and misses true ones.	😵 The student gives wrong answers and also skips some right ones.	**Totally unreliable system**.

The Truth Test: Four Combinations (MCQs Example)

✅ Sound & Complete

All provable statements are true, and all true statements can be proven.

- **Student Analogy:** Answers only correct questions and never misses any.
- **Behavior:** The ideal logical system (e.g., Propositional Logic).

❌ Not Sound but Complete

Proves everything, including false things. It finds all truths but also generates falsehoods.

- **Student Analogy:** Answers "yes" to every question, getting all truths but also many falsehoods.
- **Behavior:** Useless; results cannot be trusted.

✅ Sound but Not Complete

Only proves true statements, but misses some truths. It's reliable but limited in what it can discover.

- **Student Analogy:** Always correct, but sometimes stays silent (doesn't know some true answers).
- **Behavior:** Safe but limited (e.g., First-Order Logic with certain rule sets).

❌ Not Sound, Not Complete

Proves false things and misses true ones. It's unreliable and insufficient.

- **Student Analogy:** Gives wrong answers and skips some right ones.
- **Behavior:** Totally unreliable system.

A perfect logical system is **Sound (never wrong)** and **Complete (never miss)**.

Soundness → "Never Wrong" ✅

Correctness of derived conclusions.

Completeness → "Never Miss" 🎯

Coverage of all true conclusions.

⚖️ The Truth Test: Not Sound but Complete

Explore the implications of an inference system that can prove everything, even if some of it is false.

Complete but Untrustworthy

Imagine a system with rules so strong it can prove **every statement** – both true and false.

It's ✅ **complete** (no statement left unprovable) but ❌ **not sound** (because it proves false statements too). It "knows" everything but also "believes lies".

Mathematical Example: Peano Arithmetic

If we modify Peano Arithmetic with an extra rule like:

> "Every statement or its negation is provable,"

it becomes ✅ **complete** (can prove anything, including contradictions) but ❌ **unsound** (proves false things like "1 = 0").

Gödel's Incompleteness Connection

Gödel's theorems show that any **sound** and **consistent** mathematical system rich enough to express arithmetic **cannot be complete**. Thus, to gain completeness, one must sacrifice soundness (the guarantee of truth).

Analogy: The "Know-It-All" Student

This system is like a student who answers every math problem instantly, but sometimes says "2 + 2 = 5."

They "answer everything" (complete), but are not **trustworthy** (not sound).

✅ Sound but Not Complete

A mathematical system can be sound but not complete, a fundamental limitation revealed by Gödel's work.

💡 Definitions Refresher

- **Soundness:** Everything provable in the system is true (no false theorems).
- **Completeness:** Everything that is true can be proved in the system.

🧠 Gödel’s First Incompleteness Theorem (1931)

> Any consistent, sufficiently powerful formal system cannot be both complete and sound. If the system is sound, it must be incomplete.

1

The Statement G

You can construct a statement G within the system that says: "This statement G is not provable in this system."

2

If G is Provable...

If G were provable, then it would mean "this statement is provable," which contradicts G's claim. Thus, the system proves something false, making it ❌ **unsound**.

3

If G is True...

If G is true, then it is a true statement that the system cannot prove (to avoid unsoundness). This means the system is ✅ **incomplete**.

🧠 Gödel’s Discovery

Gödel showed that:

> If your math system is **consistent** (no contradictions)
> and **powerful enough** to do basic arithmetic,
> then it **can’t be both sound and complete** at the same time.

So, to maintain **soundness**, the system must refrain from proving G, thereby accepting its own **incompleteness**.

⚖️ The Truth Test: Soundness and Completeness

Soundness: $KB \vdash f \implies KB \models f$

If our logic system can prove something (f) from a set of facts (KB), then that something is definitely true in the real world.

"Nothing but the truth."

Completeness: $KB \models f \implies KB \vdash f$

If something (f) is true in the real world based on our facts (KB), then our logic system can find a way to prove it.

"The whole truth."

In a perfect logic system, soundness and completeness combine. This means proving something ($\vdash$) is exactly the same as it being true ($\models$).

$$KB \vdash f \quad \text{if and only if} \quad KB \models f$$

This means **if our system proves it, it's true. And if it's true, our system can prove it.**

⚖️ The Truth Oath

The Truth, The Whole Truth, and Nothing But The Truth

🔒 Soundness

"Nothing but the truth"

Every derived formula is actually true

📈 Completeness

"The whole truth"

Every true formula can be derived

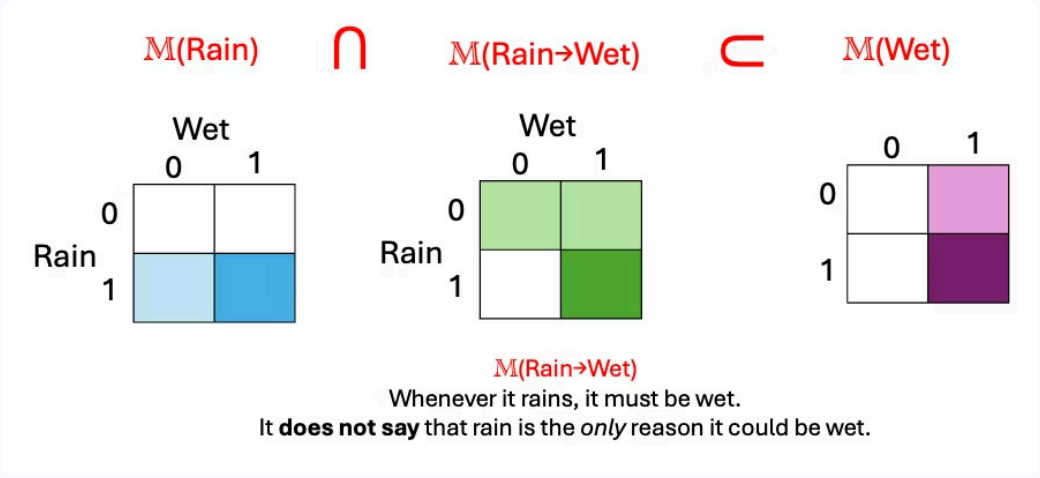
This is the oath given in sworn testimony - and exactly what we want from our inference rules!

✅ Soundness Check: Modus Ponens

Question: Is Modus Ponens sound?

$$\frac{\text{Rain}, \quad \text{Rain} \rightarrow \text{Wet}}{\text{Wet}}$$

Check: M(Rain) ∩ M(Rain → Wet) ⊆ ? M(Wet)



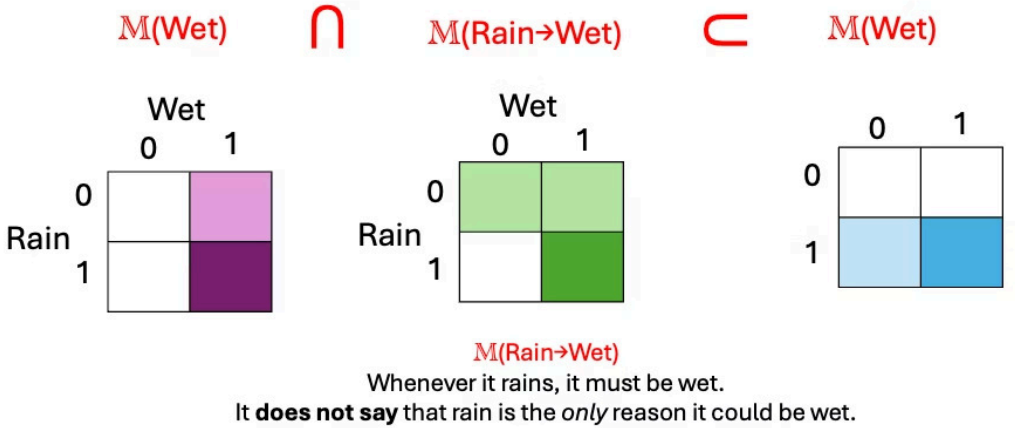
The System is Sound

❌ Unsound Rule Example

Question: Is this rule sound?

$$\frac{\text{Wet}, \quad \text{Rain} \rightarrow \text{Wet}}{\text{Rain}}$$

Check: M(Wet) ∩ M(Rain → Wet) ⊆ ? M(Rain)



⊗ The System is Unsound

✅ Sound but Not Complete: Modus Ponens

Recall that for **completeness**, inference rules should derive **all entailed formulas** (if $KB \models f$, then $KB \vdash f$).

🧠 Setup

- **Knowledge Base (KB):**

$$\text{Rain}$$
$$\text{Rain} \vee \text{Snow} \rightarrow \text{Wet}$$

- **Target formula (f):**

$$\text{Wet}$$

The only rule available is **Modus Ponens**:

$$\frac{P, \quad P \rightarrow Q}{Q}$$

From P and $P \rightarrow Q$, infer Q.

🔍 Analysis

Let's examine this scenario:

- **Semantically:**

$$KB \models \text{Wet}$$

> **"Wet"** is indeed **true** in all models that satisfy the Knowledge Base. If it's raining, and (if it's raining or snowing, then it's wet) is true, then it must be wet.

- **Syntactically:**

$$KB \nvdash \text{Wet}$$

> However, **Modus Ponens alone cannot directly derive "Wet" from these specific formulas in the KB**, because the premise $P \rightarrow Q$ needed for Modus Ponens does not directly match $\text{Rain} \vee \text{Snow} \rightarrow \text{Wet}$.

⚠ ⚠️ Conclusion

- The rule is **sound** (never wrong), but **incomplete** – it misses some truths. Therefore: **Modus Ponens alone is incomplete!**

🧩 Why Modus Ponens Fails to Ensure Completeness

While sound, Modus Ponens alone isn't always complete because its application is quite strict.

→ Strict Premise Matching

Modus Ponens requires an **exact match** for its premise (the 'P' part of $P \rightarrow Q$) to be present as a fact in the Knowledge Base (KB).

→ Atomic vs. Compound Formulas

It works perfectly when the premise is an **atomic formula** (a simple, single proposition like $Rain$). However, it struggles with **compound formulas** (propositions combined with logical connectives like $Rain \vee Snow$).

→ The Missing Link

In our example, we have $Rain$ and $(Rain \vee Snow) \rightarrow Wet$. To apply Modus Ponens, we'd need the exact compound formula $(Rain \vee Snow)$ to be present as a fact. Since we only have $Rain$, the rule cannot fire directly, even though Wet is semantically entailed.

This limitation means that some true conclusions, like Wet in our example, cannot be syntactically derived using only Modus Ponens.

⚠ Key Takeaway:

Modus Ponens is **incomplete** because it cannot handle **complex (non-atomic) premises** – it only fires when the antecedent is exactly present as a fact.

🎯 Key Takeaways: Logic Foundations

1 🧱 Logic = Syntax + Semantics + Inference

Three ingredients work together to enable reasoning

2 🌍 Models Represent Possible Worlds

Formulas compactly represent sets of models

3 📚 Knowledge Bases Constrain Reality

More formulas ▫ fewer possible worlds ▫ more specific knowledge

4 ⚡ Inference Rules Enable Direct Reasoning

Manipulate formulas without instantiating all models

🔄 The Three Relationships

✨ Entailment

KB ⊨ f

"Already knew that"



💥 Contradiction

KB ⊨ ¬f

"Don't believe that"

🤔 Contingency

Neither entailed nor contradicted

"Learned something new"



🚀 From Logic to AI Applications

1

🧱 Propositional Logic

Foundation we've built today

2

🌟 First-Order Logic

More expressive, handles objects and relations

3

🤖 Smart Assistants

Real-world reasoning applications

Logic provides the **expressiveness** that pure probability and machine learning haven't yet recouped!

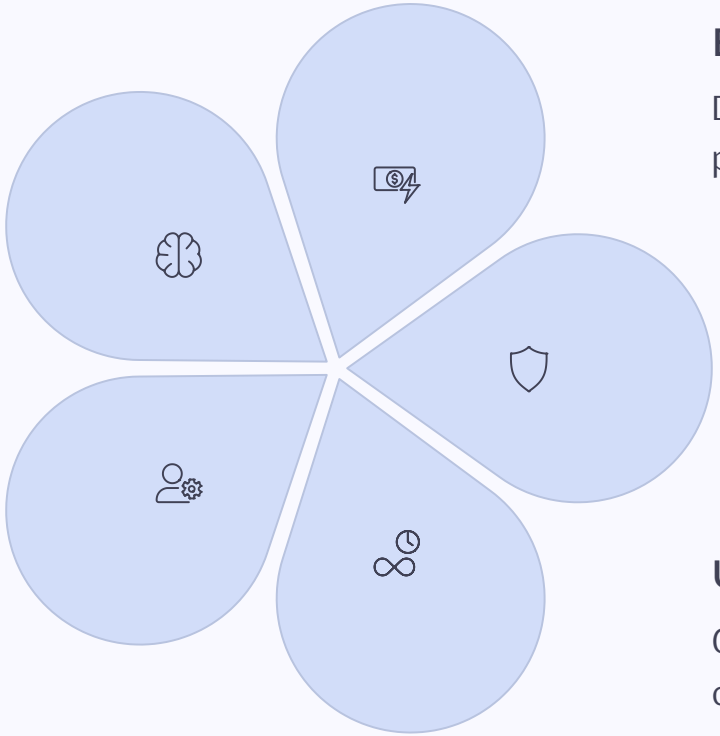
The Power of Logical Inference

Compact Representation

One formula represents many models

Mechanical Process

Computers can apply rules automatically



Efficient Reasoning

Derive conclusions without checking all possibilities

Guaranteed Correctness

Sound inference rules never produce false conclusions

Unlimited Expressivity

Can represent complex relationships and constraints